Microservices Interview Notes (Printable)

Detailed Q&A covering service-based + product-based microservices interviews (architecture, data, resilience, observability, security, DevOps).

Focus: Concepts + Trade-offs + Real-world patterns

Includes: Saga, Outbox, Idempotency, Observability, K8s, Mesh

Use: Revise + speakable interview answers

Table of Contents

1 What are Microservices? Why do we use them?

Architecture | Trade-offs | Team scaling

Core idea

Microservices is an architectural style where an application is built as a **suite of small, independently deployable services**. Each service owns a **single business capability** (example: Order, Payment, Inventory) and communicates with others via APIs/events.

Why companies adopt microservices

- **Independent deployments:** deploy Payment without redeploying Order.
- **Independent scaling:** scale only high-traffic services (Search, Checkout).
- **Team autonomy:** multiple teams deliver in parallel with clear ownership.
- **Fault isolation:** with resilience patterns, one service failure does not crash all.

Cost / complexity introduced

- **Distributed systems problems:** latency, partial failures, retries, network partitions.
- **Data consistency challenges:** multi-service transactions are difficult.
- **Ops overhead:** CI/CD, monitoring, on-call, infra maturity required.

> **Interview line:** "Microservices optimize for independent delivery and organizational scale, at the cost of distributed-systems complexity."

> **Speakable (30–45s):** "Microservices split an application into small services around business capabilities. Each service can be deployed and scaled independently. This helps large teams ship faster and reduces blast radius. But it adds distributed system complexity—network failures, observability, and eventual consistency—so it needs good DevOps and design."

2 Microservices vs Monolith: When would you choose each?

Decision making | Trade-offs

Monolith is a good choice when

- Small team/early stage, fast iteration needed
- Domain boundaries not clear yet
- You want simpler debugging/testing/deployment
- You need strong consistency with single DB transactions

Microservices are a good choice when

- Multiple teams need independent releases
- Clear bounded contexts exist
- Different parts need different scaling patterns
- Org has CI/CD, monitoring, on-call readiness

> **Common path:** Monolith → Modular Monolith → Microservices (split when boundaries stabilize).

> **Speakable:** "Monolith is simpler and great early. Microservices help when teams and domain grow—independent deploy and scaling—but require mature ops and careful service boundaries."

3 What is a “bounded context” and why is it important?

DDD | Service boundaries

Meaning

A bounded context is a boundary where a domain model has a **consistent meaning**. Same word can mean different things in different contexts.

Example: “Customer” in Order service may mean shipping contact; in Payment it may mean billing identity and fraud signals.

Why it matters

- Helps define service boundaries around business capability
- Reduces “chatty” cross-service calls
- Avoids shared DB and tight coupling
- Prevents “distributed monolith”

> **Rule of thumb:** Split by business capabilities, not by technical layers (controller/service/repo).

4 How do microservices communicate? Sync vs Async

REST | gRPC | Kafka | Queues

Synchronous communication (HTTP/gRPC)

Caller waits for response. Good for query-like reads or immediate validation.

- **Pros:** simple flow, easy to reason about, immediate result
- **Cons:** tight runtime coupling, higher latency, cascading failures, thread blocking

Asynchronous communication (events/queues)

Producer publishes an event/message. Consumers process later.

- **Pros:** decoupling, resilience, buffers spikes, better throughput
- **Cons:** eventual consistency, duplicates, ordering complexity, harder debugging

> **Interview line:** “Sync for immediate reads; async for workflows and integration to reduce coupling.”

5 What is Service Discovery?

Eureka Consul Kubernetes DNS

Problem it solves

Service instances scale up/down. IPs change. Hardcoding endpoints breaks. Service discovery provides a dynamic mapping from **service name → instances**.

Patterns

- **Client-side discovery:** client queries registry and load balances (Eureka style).
- **Server-side discovery:** proxy/load balancer routes to instances (common in Kubernetes).

> **Kubernetes note:** K8s Services + DNS usually handle discovery; clients call a stable service name.

6 API Gateway: Why and what responsibilities?

Edge Routing Security

Why it exists

Clients should not call many internal services directly. Gateway provides a single entry point and hides internal topology.

Typical responsibilities

- Routing and request forwarding
- Authentication token validation
- Rate limiting and throttling
- Request/response transformation
- Caching for common reads
- Sometimes aggregation (carefully)

> **Warning:** Do not put business logic in gateway. Keep it thin.

7 Database per service vs shared database: what's correct?

Data ownership | Coupling

Ideal principle

Each service should **own its data** (database-per-service or at least schema-per-service) to enable independent deploy and evolution.

Why shared DB is harmful

- Schema changes break multiple services
- Teams become tightly coupled and blocked
- Services start reading each other's tables directly
- Independent deployment becomes difficult

> **Real-world migration:** Some systems start shared for legacy reasons; they gradually split by schema and introduce APIs/events.

8 How do you maintain data consistency across services?

Eventual consistency | Saga | Outbox

Why it's hard

In monoliths, one DB transaction can update everything. In microservices, data is distributed; network calls can fail, so global ACID is difficult.

Common solutions

- **Eventual consistency:** accept that state converges over time.
- **Saga pattern:** step-by-step workflow with compensation for failures.
- **Outbox pattern:** reliable event publishing with DB changes.
- **Idempotency:** safe retries without incorrect duplicates.

> **Interview line:** "We avoid 2PC; we use Saga + Outbox + idempotent consumers to achieve reliable eventual consistency."

9 Explain Saga Pattern with example

Distributed workflow | Compensation

Example: Place Order

1. Order created (PENDING)
2. Payment charged
3. Inventory reserved
4. Shipping scheduled

If inventory reservation fails after payment succeeds, we trigger **compensation** like refund payment and cancel order.

Types

- **Choreography:** services react to events (no central controller). Simple components but hard to trace at scale.
- **Orchestration:** a workflow orchestrator commands steps and handles retries/compensations. Easier visibility but orchestrator is critical component.

> **Keyword:** Compensating transaction (undo action) is the core of Saga.

10 What is the Outbox pattern and why it matters?

Dual write problem | Reliable events

The "dual-write" problem

If a service updates its DB and separately publishes an event, either step can fail, causing inconsistent state.

Outbox solution

- In the same DB transaction: write business data + write an outbox event record (payload, status).
- A background publisher reads the outbox table and publishes to Kafka/queue.
- Mark outbox record as SENT after successful publish.

> **Benefit:** Strong reliability with at-least-once delivery; consumers must handle duplicates (idempotency).

11 How do you handle duplicate messages?

At-least-once | Idempotency

Why duplicates happen

Retries and at-least-once messaging mean the same message can be delivered more than once.

Strategies

- **Idempotent consumers:** processing the same event twice yields same final state.
- **Dedup store:** maintain processed message IDs for a time window.
- **DB uniqueness constraints:** enforce one-time operations using unique keys (transactionId, eventId).

> **Best practice:** Assume duplicates are normal; design for them.

12 CAP theorem & trade-offs in microservices

Consistency | Availability | Partitions

Concept

CAP states that in the presence of a network partition (P), a distributed system can choose either Consistency (C) or Availability (A) fully, but not both simultaneously.

Implications

- **CP systems:** prefer consistency; may reject requests during partition.
- **AP systems:** prefer availability; may return stale data; reconcile later (eventual consistency).

> **Interview insight:** Microservices often accept eventual consistency for availability and scalability, but critical flows may require stricter consistency.

13 Resilience patterns: Circuit Breaker, Retry, Timeout, Bulkhead

Resilience | Cascading failure

Timeout

Always set timeouts. Without timeouts, threads can block indefinitely, exhausting thread pools and connection pools.

Retry

Use retries for transient failures (timeouts/503). Use exponential backoff + jitter. Avoid retrying non-idempotent calls unless you have idempotency keys.

Circuit Breaker

When failures cross a threshold, breaker opens and requests fail fast for a cool-down period. Prevents hammering a failing dependency and reduces cascading failures.

Bulkhead

Isolate resources so failure in one dependency does not take down the whole service (separate thread pools, connection pools, rate limits).

> **Practical combo:** Timeout + controlled retry + circuit breaker + bulkheads.

14 Distributed Tracing: how do you debug across services?

traceId | OpenTelemetry

What it is

Distributed tracing tracks a request as it flows across services using a **traceId** and **spanId** per hop.

Why it matters

- Pinpoints where latency occurs (which span is slow)
- Finds the failing hop in long call chains
- Correlates logs and metrics with traceId

> **Tools:** OpenTelemetry + Jaeger/Zipkin/Tempo, or Datadog/NewRelic in many companies.

15 Observability: Metrics vs Logs vs Traces

Monitoring Debugging

Metrics

Aggregated numbers over time (RPS, error rate, latency p95/p99). Best for dashboards and alerts.

Logs

Discrete events (errors, info, debug). Best to understand details: inputs, decisions, stack traces.

Traces

End-to-end view of a request journey across services. Best to diagnose slow/failing dependencies.

> **Mature approach:** Alert on metrics → investigate via traces → confirm details via logs.

16 What is centralized logging and why needed?

ELK Correlation

Why needed

Services run across many nodes/containers. Centralized logging collects logs into a single place for search, alerting, and correlation.

Best practices

- Structured logs (JSON) for query-friendly fields
- Correlation IDs / traceId in every log line
- Log levels and sampling to control cost/noise

> **Common stack:** ELK/EFK or cloud-native logging.

17 API versioning strategies

Backward compatibility | Deprecation

Ways to version

- URL: `/v1/orders`
- Header: `Accept: application/vnd.company.v1+json`
- Query: `?version=1` (least preferred)

What matters most

- Prefer backward-compatible changes (add optional fields)
- Version only for breaking changes
- Have clear deprecation policy and migration plan

18 How do you secure microservices?

OAuth2/OIDC | JWT | mTLS

Core security layers

- **Authentication (AuthN):** who are you?
- **Authorization (AuthZ):** what can you do?
- **Transport security:** encryption in transit (TLS/mTLS)
- **Secrets management:** store secrets outside code

Common approach

- OAuth2/OIDC identity provider issues JWT tokens
- Gateway validates token and forwards identity claims
- Services enforce authorization based on roles/scopes
- Service-to-service: mTLS (mesh) or client-credentials tokens

> **Also mention:** least privilege, rate limiting, audit logs, secret rotation.

19 What is “Zero Trust” in microservices?

Security model | Policy

Zero Trust assumes the internal network is not safe. Every request must be authenticated and authorized, even inside a cluster.

Practical implementation

- mTLS between services
- Identity for services and policy-based access control
- Least privilege permissions and segmentation

20 Configuration management in microservices

ConfigMaps | Secrets | Feature flags

Need

Many services across dev/qa/prod need environment-specific configuration without code changes.

Best practices

- Externalize configs (no hard-coding)
- Separate secrets from configs (vault/K8s secrets)
- Version config changes and audit them
- Use feature flags for gradual rollouts

21 Deployment strategies for microservices

Blue/Green | Canary | Rolling

Strategies

- **Rolling:** replace pods gradually
- **Blue/Green:** switch traffic between two environments
- **Canary:** release to small % and monitor metrics
- **Feature flags:** deploy code but enable feature later

> **Product-based interviews:** mention “progressive delivery” + “metrics-based rollback”.

22 Containerization & orchestration (Docker + Kubernetes)

Docker K8s Scaling

Docker

Packages app + dependencies into a container image so it runs consistently across environments.

Kubernetes

Manages containers at scale:

- Self-healing (restart failed pods)
- Horizontal scaling (replicas)
- Service discovery + load balancing
- Rolling deployments
- ConfigMaps/Secrets

> **Key objects:** Pod, Deployment, Service, Ingress, ConfigMap, Secret.

23 What is a Service Mesh (Istio/Linkerd)?

Sidecar mTLS Traffic mgmt

A service mesh adds a data plane (proxies) to handle service-to-service communication concerns without writing them in each service.

What mesh provides

- mTLS and secure identities
- Traffic routing (canary, retries, timeouts)
- Observability (telemetry, tracing headers)
- Policy enforcement

> **Trade-off:** Adds infra complexity; needs operational maturity.

24 Handling failures and cascading failure prevention

Timeouts | Circuit breaker | Bulkhead

What is cascading failure?

One slow/down dependency causes upstream services to block and exhaust resources, spreading the outage.

Prevention techniques

- Timeouts everywhere
- Circuit breakers to fail fast
- Bulkheads to isolate resources
- Fallbacks/graceful degradation
- Async workflows to reduce tight coupling

25 How do you test microservices?

Unit | Integration | Contract | E2E

Testing layers

- **Unit tests:** fastest, business logic
- **Integration tests:** DB/messaging/external dependencies (often using containers)
- **Contract tests:** ensure API compatibility between services
- **E2E tests:** few, critical flows only (expensive/slow)
- **Performance tests:** load and latency checks

> **Strategy:** Many unit + contract tests, limited E2E.

26 What is Consumer-Driven Contract Testing (CDC)?

Pact | Compatibility

Consumers define expectations for provider APIs (response schema, status codes, required fields). Providers verify they satisfy these contracts during CI.

Why it's valuable

- Services deploy independently
- Prevents breaking changes from reaching production
- Reduces reliance on heavy E2E test suites

27 Schema evolution in events (Kafka) / backward compatibility

Avro/Protobuf | Schema registry | Compatibility

Why it matters

Many consumers may read an event. If producer changes the schema, old consumers can break.

Safe evolution rules

- Prefer **additive** changes (add optional fields)
- Avoid removing/renaming fields without versioning
- Use schema registry with compatibility checks

> **Also important:** ordering (partition keys), replay handling, idempotent consumers.

28 Handling “chatty” communication and performance problems

Latency | Boundaries | Caching

Symptoms

- Many sequential network calls increase latency
- Higher failure probability (more hops)
- More infra cost

Solutions

- Improve service boundaries to reduce cross-calls
- Use async events for workflows
- Caching for read-heavy data
- Read models / CQRS for fast queries
- gRPC for efficient internal communication (optional)

29 CQRS: What and why?

Read model | Write model | Event-driven

Definition

CQRS separates **commands** (writes) from **queries** (reads), often using different models optimized for each.

Why use it

- Read-heavy systems need optimized, denormalized read models
- Scale reads and writes independently

Trade-offs

- More complexity and eventual consistency
- Need pipelines to build and maintain read models

30 Common microservices anti-patterns

Distributed monolith | Shared DB

- **Shared database** across multiple services
- **Distributed monolith:** many synchronous dependencies; deploys are “independent” only on paper
- **Nano-services:** too many tiny services leading to complexity and latency
- **No observability:** no tracing/metrics; hard to debug
- **Shared libraries that lock versions:** forces coordinated releases
- **Poor boundaries:** split by technical layers, not domain
- **No resilience:** missing timeouts/circuit breakers

> **Interview line:** “Microservices without good boundaries and resilience become a distributed monolith.”

31 What is Idempotency and why is it critical?

Retries | Safe processing

Definition

An operation is idempotent if performing it multiple times results in the same final outcome as performing it once.

Why it matters

- Network timeouts cause clients to retry
- Message delivery is often at-least-once
- Services may crash after doing work but before acknowledging

Common implementations

- **Idempotency key:** client sends a unique key; server stores result for that key
- **Unique constraints:** enforce one-time side effects (transactionId uniqueness)
- **Dedup store:** track processed event IDs

> **Example:** Payment charge must be idempotent; otherwise retries may double-charge.

32 How do you handle rate limiting and throttling?

429 | Token bucket | Protection

Why needed

- Protects services from overload
- Prevents abuse and noisy neighbors
- Stabilizes latency under spikes

Where to apply

- API Gateway / Ingress (most common)
- Service mesh policies
- Per-service internal rate limits for critical resources

Algorithms

- Token bucket / leaky bucket
- Fixed window / sliding window counters

> **Response:** Use HTTP 429 (Too Many Requests) and optionally `Retry-After`.

33 What is Backpressure in microservices?

Flow control | Queues

Meaning

Backpressure is flow control: when downstream is slow, upstream must slow down or reject work to avoid overload and collapse.

How to handle

- Limit queue sizes; reject early when full
- Consumer rate control (pause/resume partitions)
- Concurrency limits and bounded thread pools
- Load shedding (drop non-critical work)

> **Goal:** keep system stable rather than accepting unlimited work and failing catastrophically.

34 How do you do “graceful shutdown”?

Rolling deploy | Readiness | No dropped requests

Why it matters

During deployments, Kubernetes may terminate pods. Without graceful shutdown, in-flight requests are dropped and clients see errors.

Steps

- Stop accepting new requests (fail readiness probe)
- Allow in-flight requests to complete (grace period)
- Close resources cleanly (connections, threads)
- Ensure load balancer stops routing to the instance

> **Keywords:** readiness vs liveness probes, termination grace period, drain connections.

35 How do you design microservices for high availability?

HA | Multi-zone | Resilience

Core practices

- Run multiple replicas behind load balancer
- Health checks (readiness/liveness)
- Deploy across zones/regions to tolerate failures
- Time-outs + circuit breakers + bulkheads
- Database replication, backups, tested restore
- Chaos testing / failure drills (advanced)

> **Speakable:** “HA is redundancy plus quick failover. We run multiple replicas across zones, use proper health checks, safe deploy strategies, and resilience patterns. For data, we use replication and backups, and verify recovery through drills.”

How to use these notes: Read Q1–Q10 daily for fundamentals, then rotate other sections. Practice speaking the “Speakable” answers aloud.

Tip: For product-based interviews, always add trade-offs and failure scenarios.